

Cache Craftiness for Fast Multicore Key-Value Storage

Yandong Mao (MIT), Eddie Kohler
(Harvard), Robert Morris (MIT)

Let's build a fast key-value store

- KV store systems are important
 - Google Bigtable, Amazon Dynamo, Yahoo! PNUTS
- Single-server KV performance matters
 - Reduce cost
 - Easier management
- Goal: fast KV store for single multi-core server
 - Assume all data fits in memory
 - Redis, VoltDB

Feature wish list

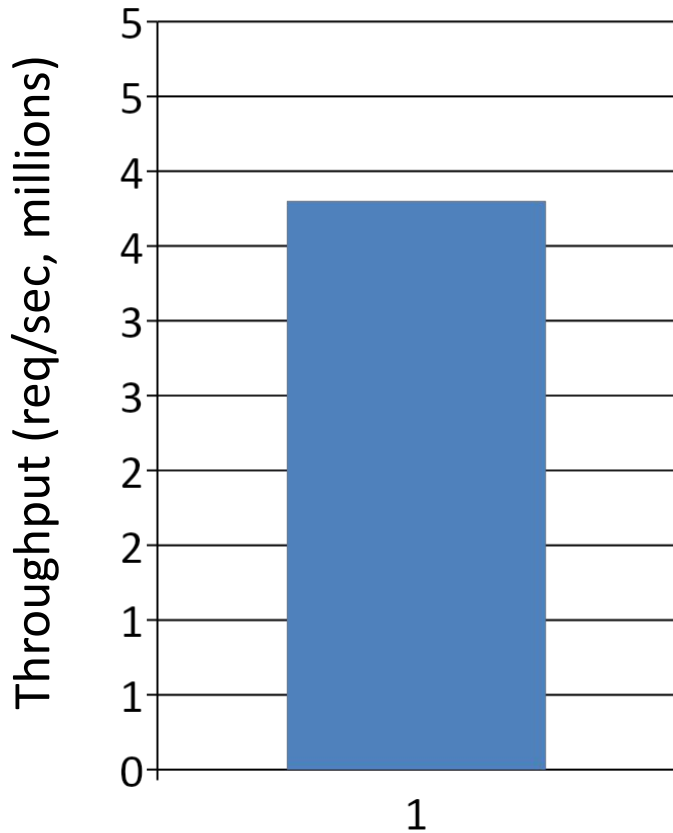
- Clients send queries over network
- Persist data across crashes
- Range query
- Perform well on **various workloads**
 - Including hard ones!

Hard workloads

- Skewed key popularity
 - Hard! (Load imbalance)
- Small key-value pairs
 - Hard!
- Many puts
 - Hard!
- Arbitrary keys
 - String (e.g. www.wikipedia.org/...) or integer
 - Hard!

First try: fast binary tree

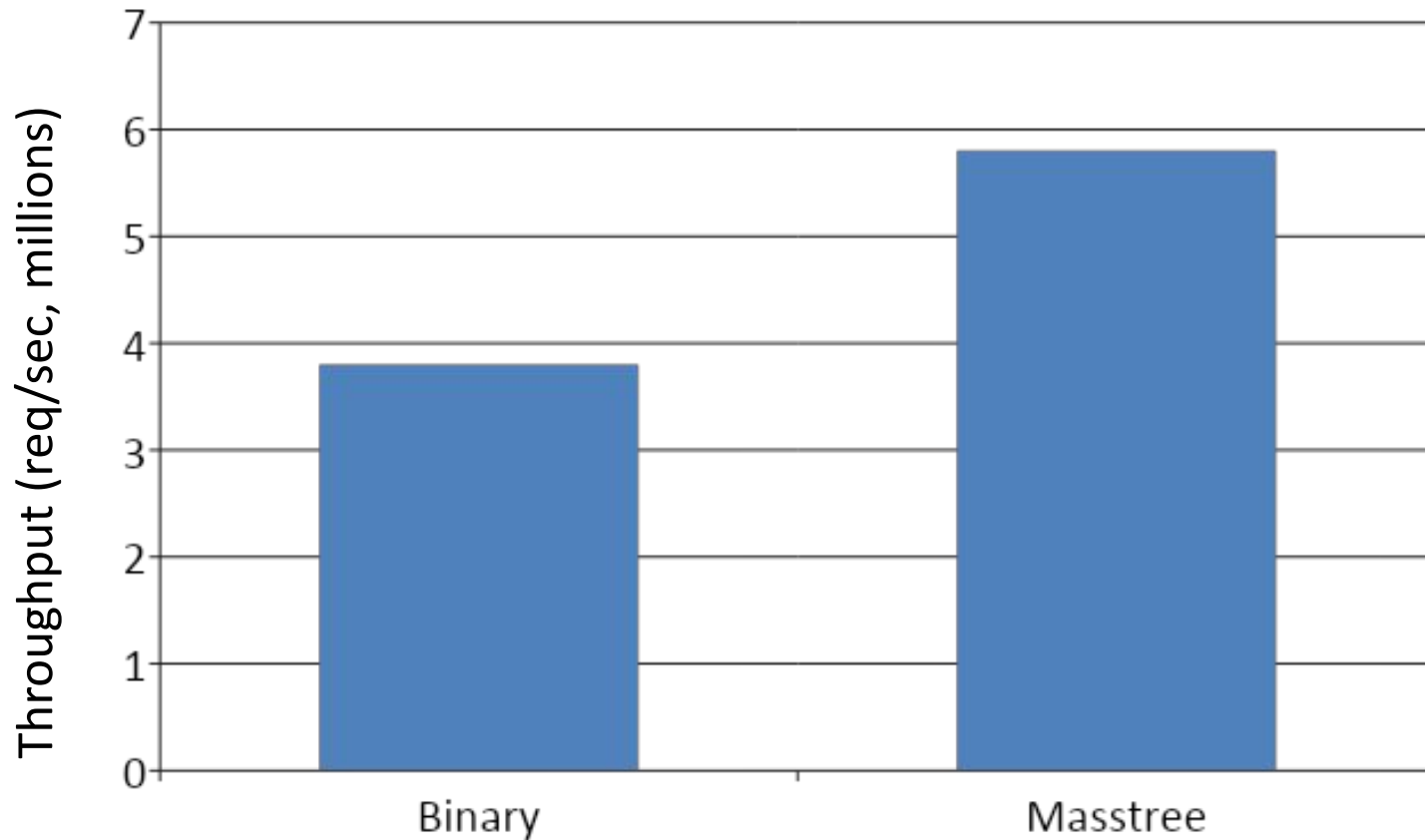
140M short KV, put-only, @16 cores



- Network/disk not bottlenecks
 - High-BW NIC
 - Multiple disks
- 3.7 million queries/second!
- Better?
- What bottleneck remains?
 - **DRAM!**

Cache craftiness goes 1.5X farther

140M short KV, put-only, @16 cores



Cache-craftiness: careful use of cache and memory

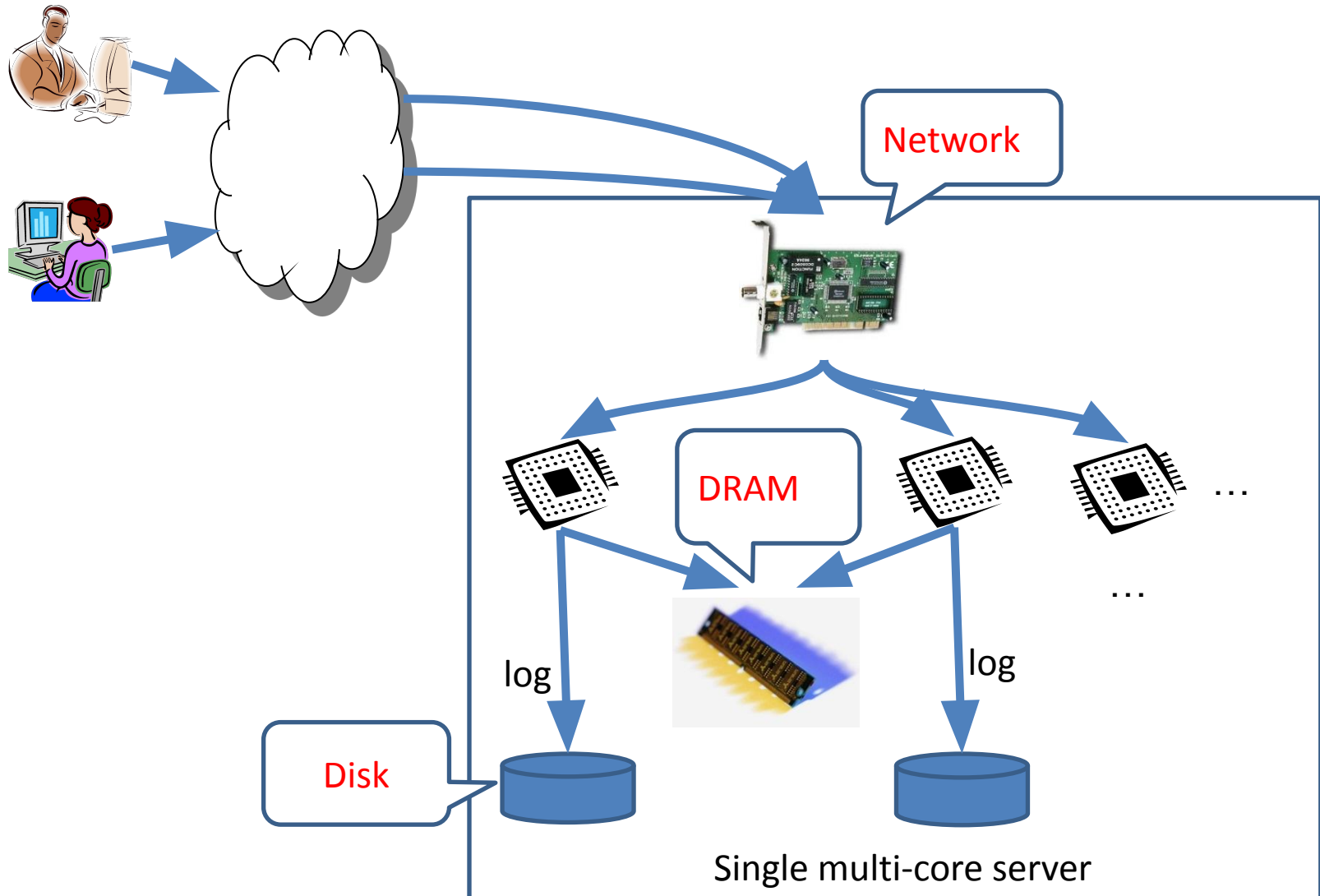
Contributions

- Masstree achieves millions of queries per second across various **hard** workloads
 - Skewed key popularity
 - Various read/write ratios
 - Variable relatively long keys
 - Data >> on-chip cache
- New ideas
 - Trie of B+ trees, permuter, etc.
- Full system
 - New ideas + best practices (network, disk, etc.)

Experiment environment

- A 16-core server
 - three active DRAM nodes
- Single 10Gb Network Interface Card (NIC)
- Four SSDs
- 64 GB DRAM
- A cluster of load generators

Potential bottlenecks in Masstree



NIC bottleneck can be avoided

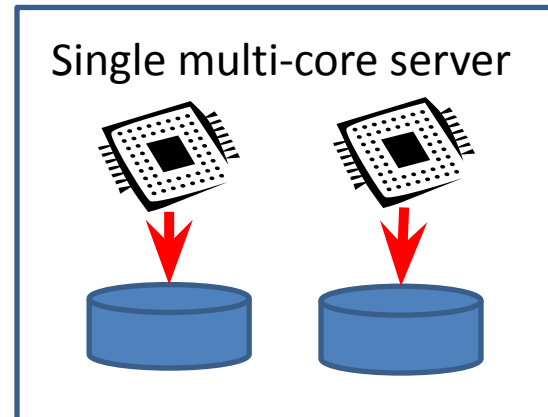
- Single 10Gb NIC
 - Multiple queue, scale to many cores
 - Target: 100B KV pair => 10M/req/sec
- Use network stack efficiently
 - Pipeline requests
 - Avoid copying cost

Disk bottleneck can be avoided

- 10M/puts/sec => 1GB logs/sec!
- Single disk

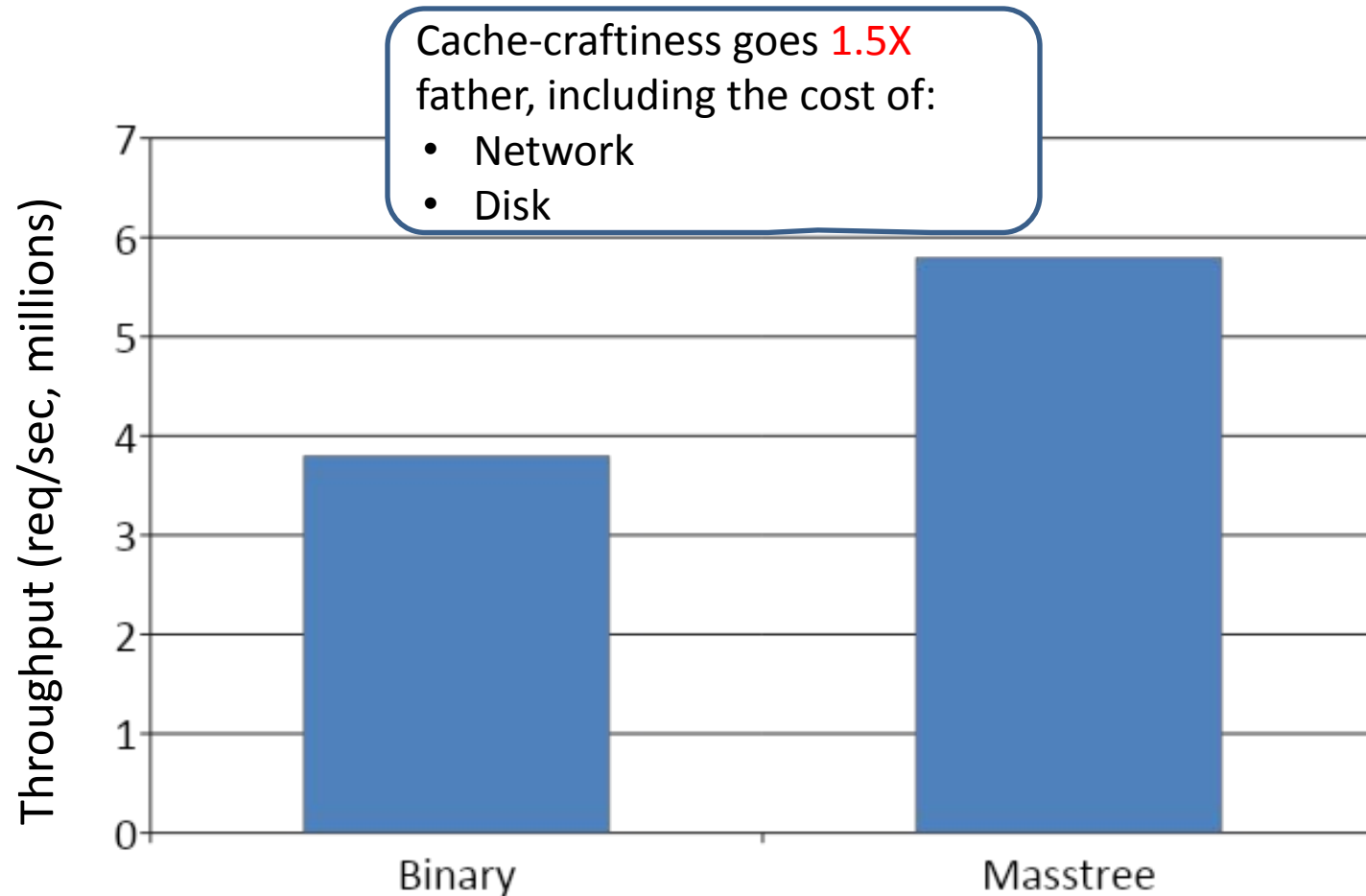
	Write throughput	Cost
Mainstream Disk	100-300 MB/sec	1 \$/GB
High performance SSD	up to 4.4GB/sec	> 40 \$/GB

- Multiple disks: split log
 - See paper for details



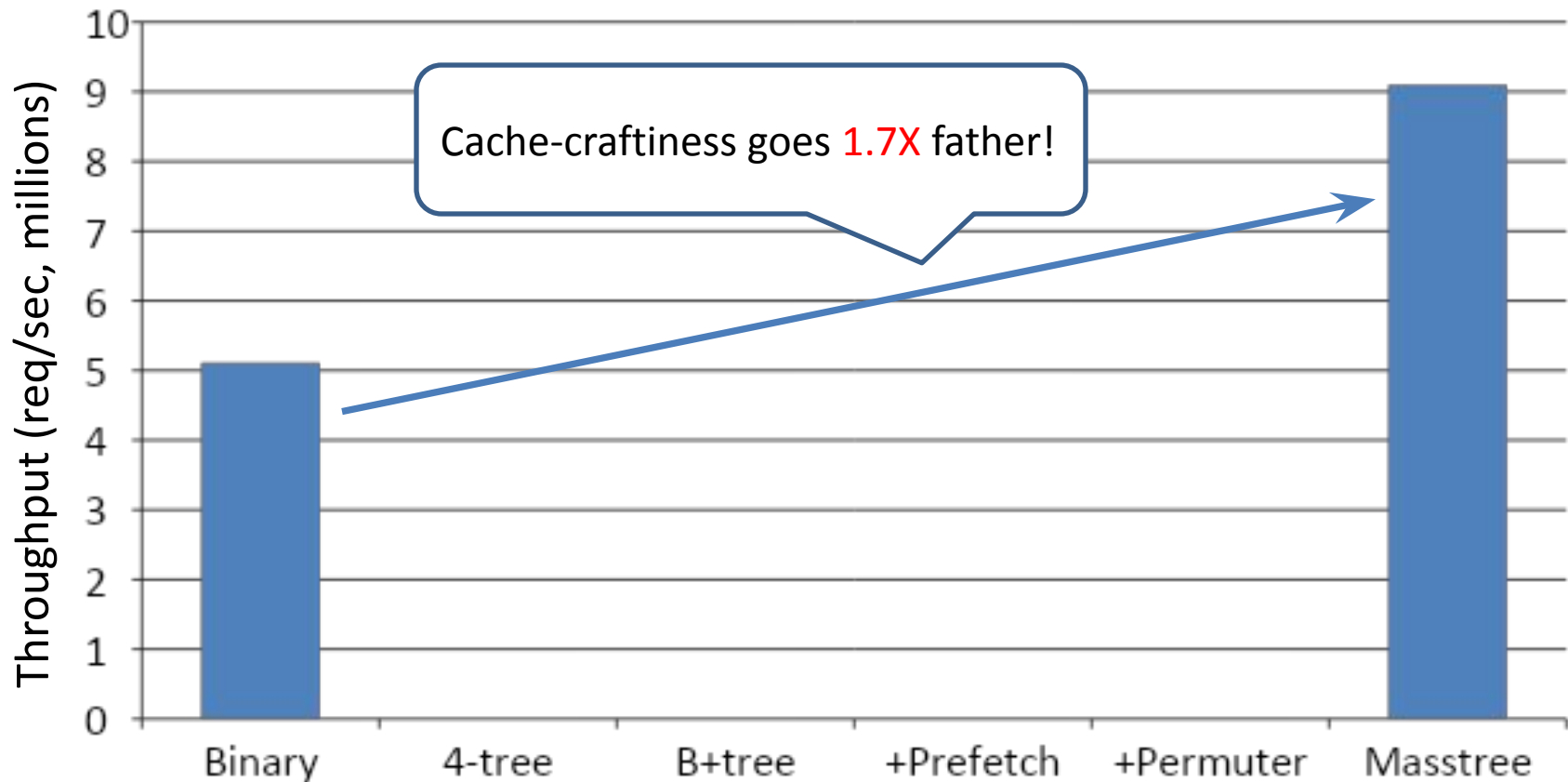
DRAM bottleneck – hard to avoid

140M short KV, put-only, @16 cores



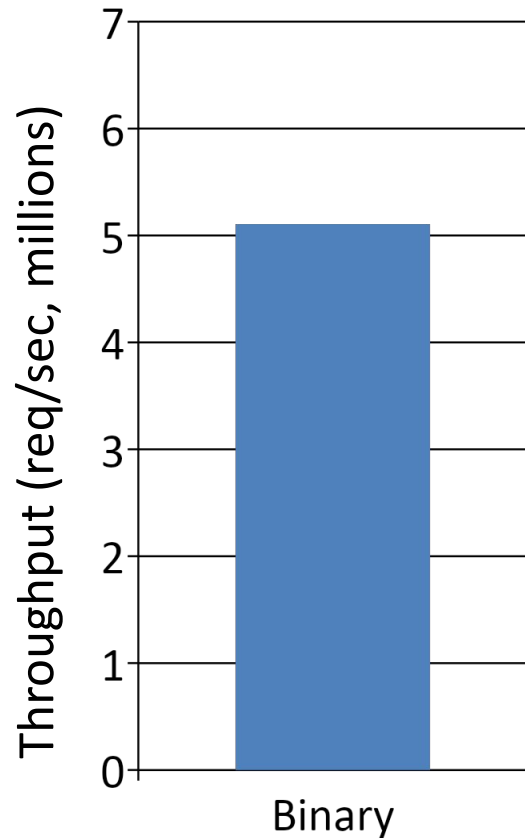
DRAM bottleneck – w/o network/disk

140M short KV, put-only, @16 cores

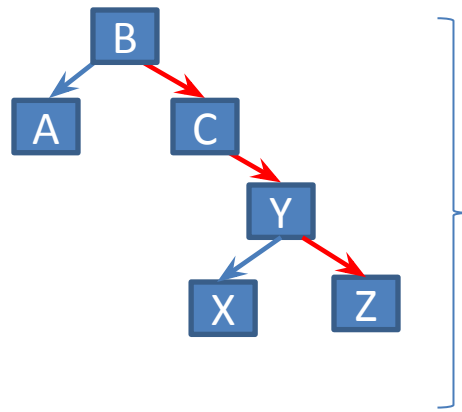


DRAM latency – binary tree

140M **short** KV, put-only, @16 cores



VoltDB



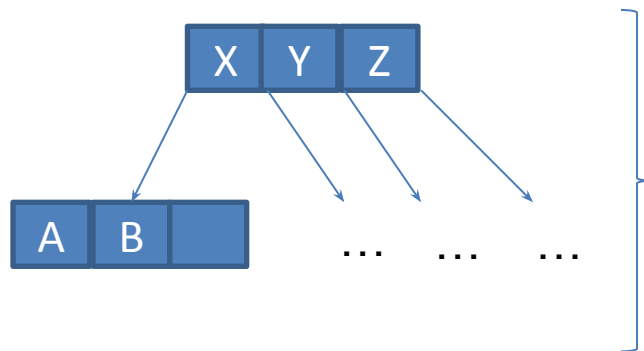
$O(\log_2 N)$ serial
DRAM latencies!

10M keys =>

2.7 us/lookup
380K lookups/core/sec

DRAM latency – Lock-free 4-way tree

- Concurrency: same as binary tree
- **One** cache line per node => 3 KV / 4 children



½ levels as binary tree

½ DRAM latencies as binary tree

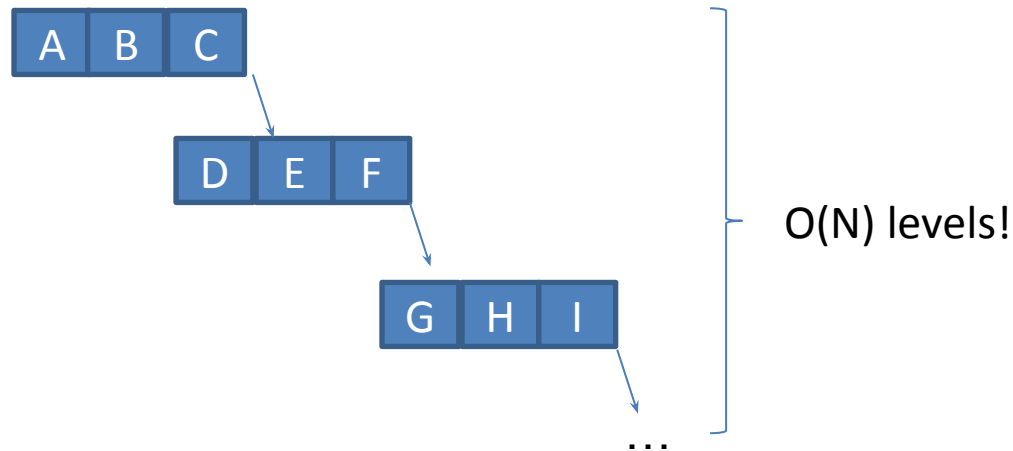
4-tree beats binary tree by 40%

140M short KV, put-only, @16 cores



4-tree may perform terribly!

- Unbalanced: $O(N)$ serial DRAM latencies
 - e.g. sequential inserts



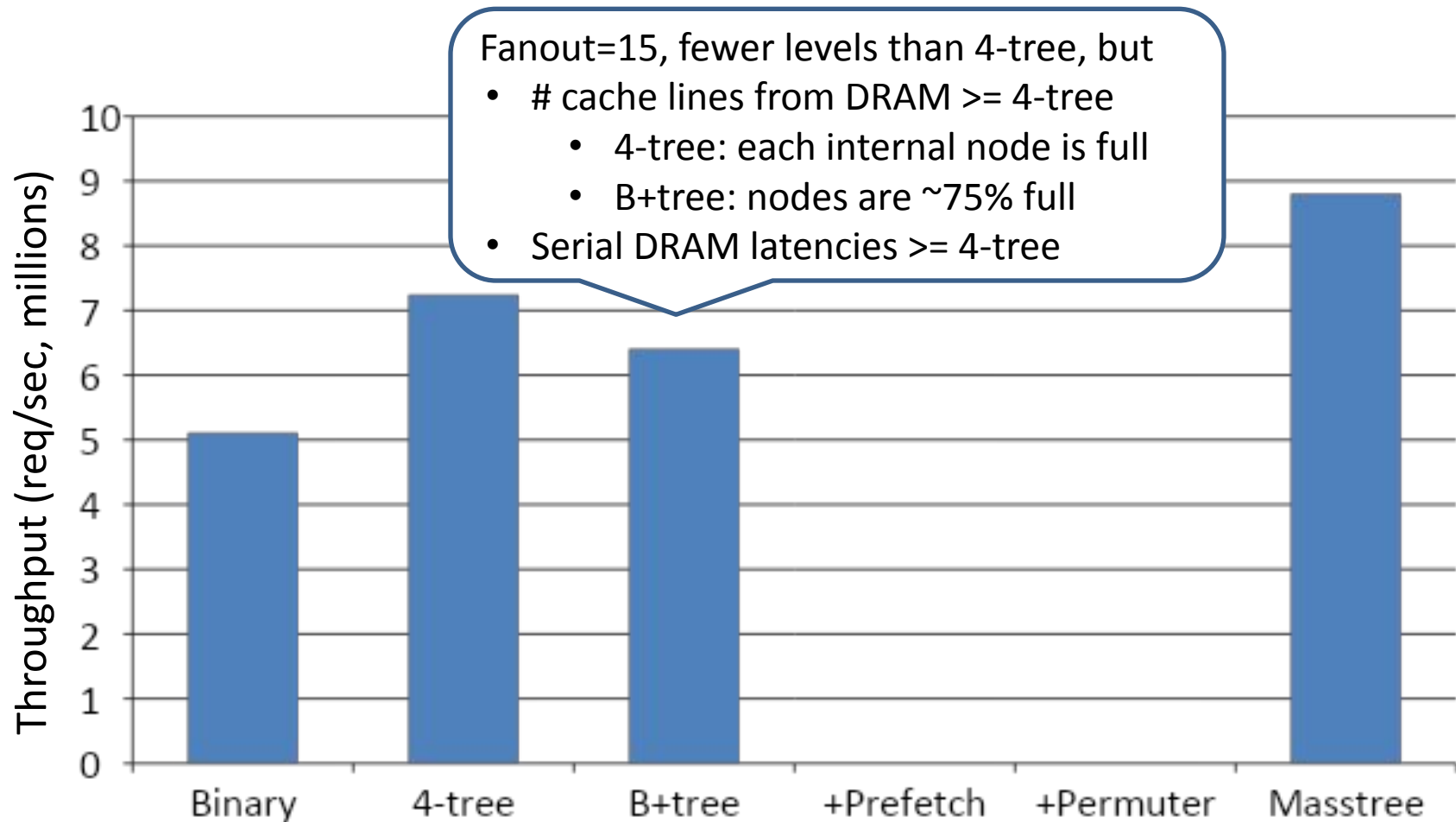
- Want balanced tree w/ wide fanout

B+tree – Wide and balanced

- Balanced!
- Concurrent main memory B+tree [OLFIT]
 - Optimistic concurrency control: version technique
 - Lookup/scan is lock-free
 - Puts hold ≤ 3 per-node locks

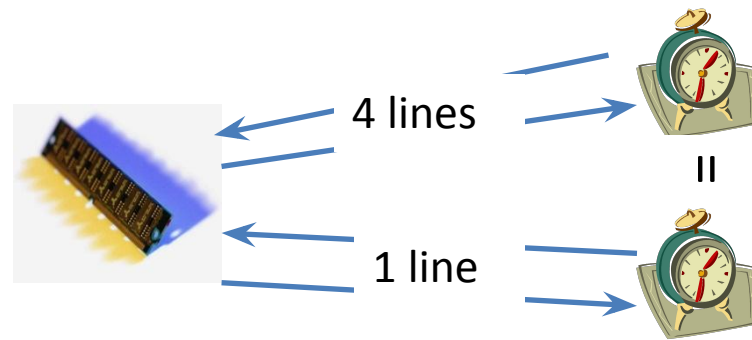
Wide fanout B+tree is 11% slower!

140M short KV, put-only



B+tree – Software prefetch

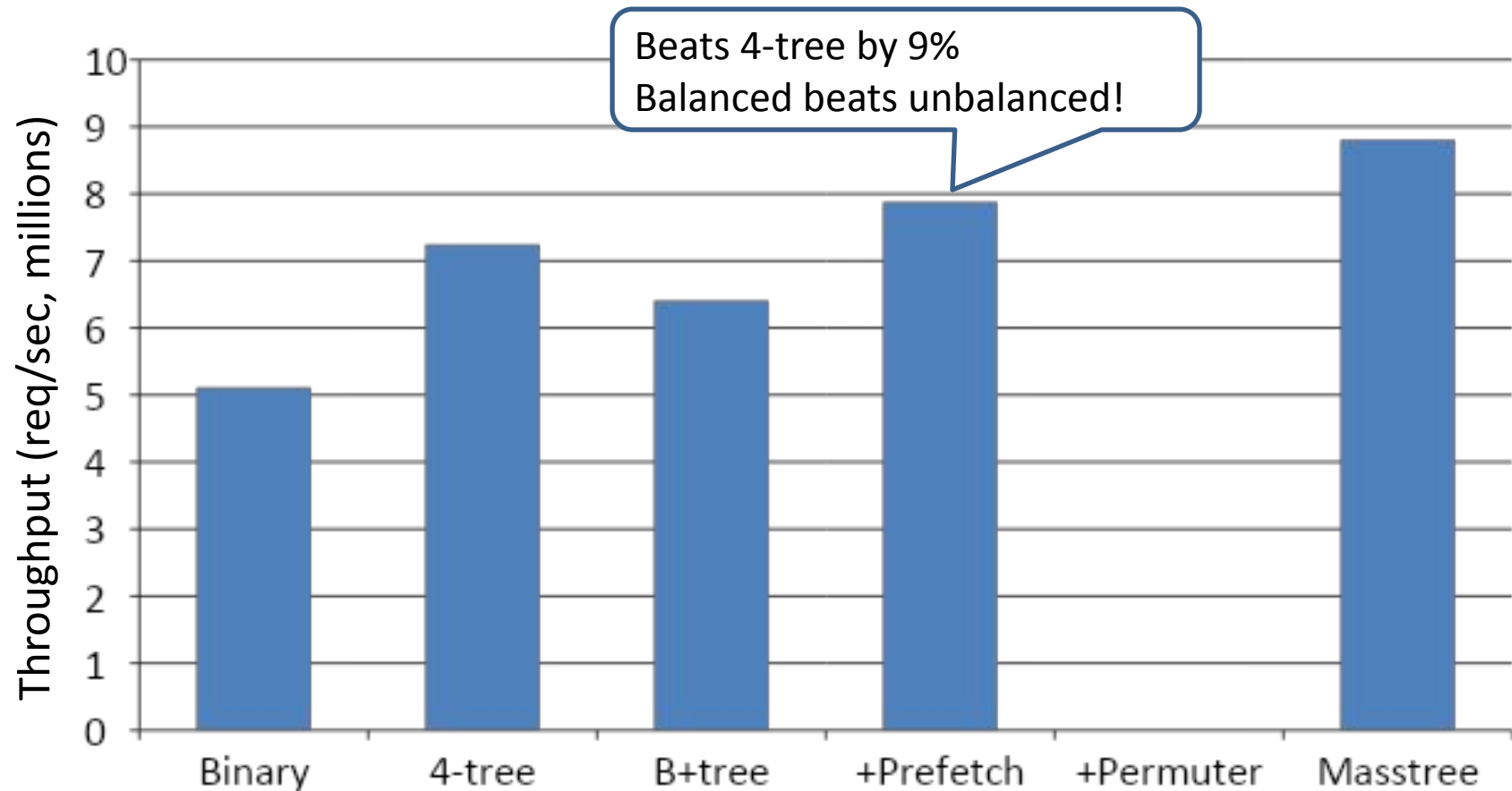
- Same as [pB⁺-trees]



- Masstree: B+tree w/ fanout 15 => 4 cache lines
- Always prefetch whole node when accessed
- Result: one DRAM latency per node vs. 2, 3, or 4

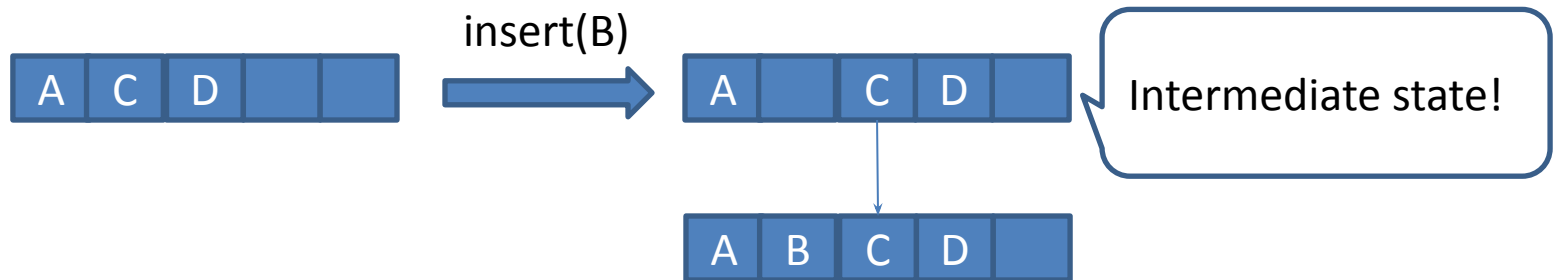
B+tree with prefetch

140M short KV, put-only, @16 cores



Concurrent B+tree problem

- Lookups retry in case of a concurrent insert

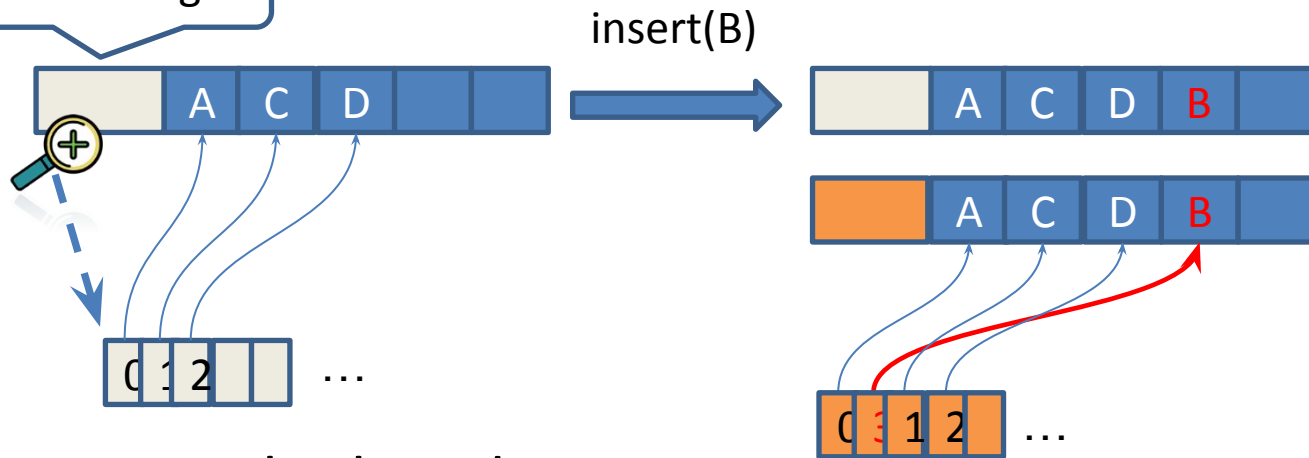


- Lock-free 4-tree: not a problem
 - keys do not move around
 - but unbalanced

B+tree optimization - Permuter

- Keys stored unsorted, define order in tree nodes

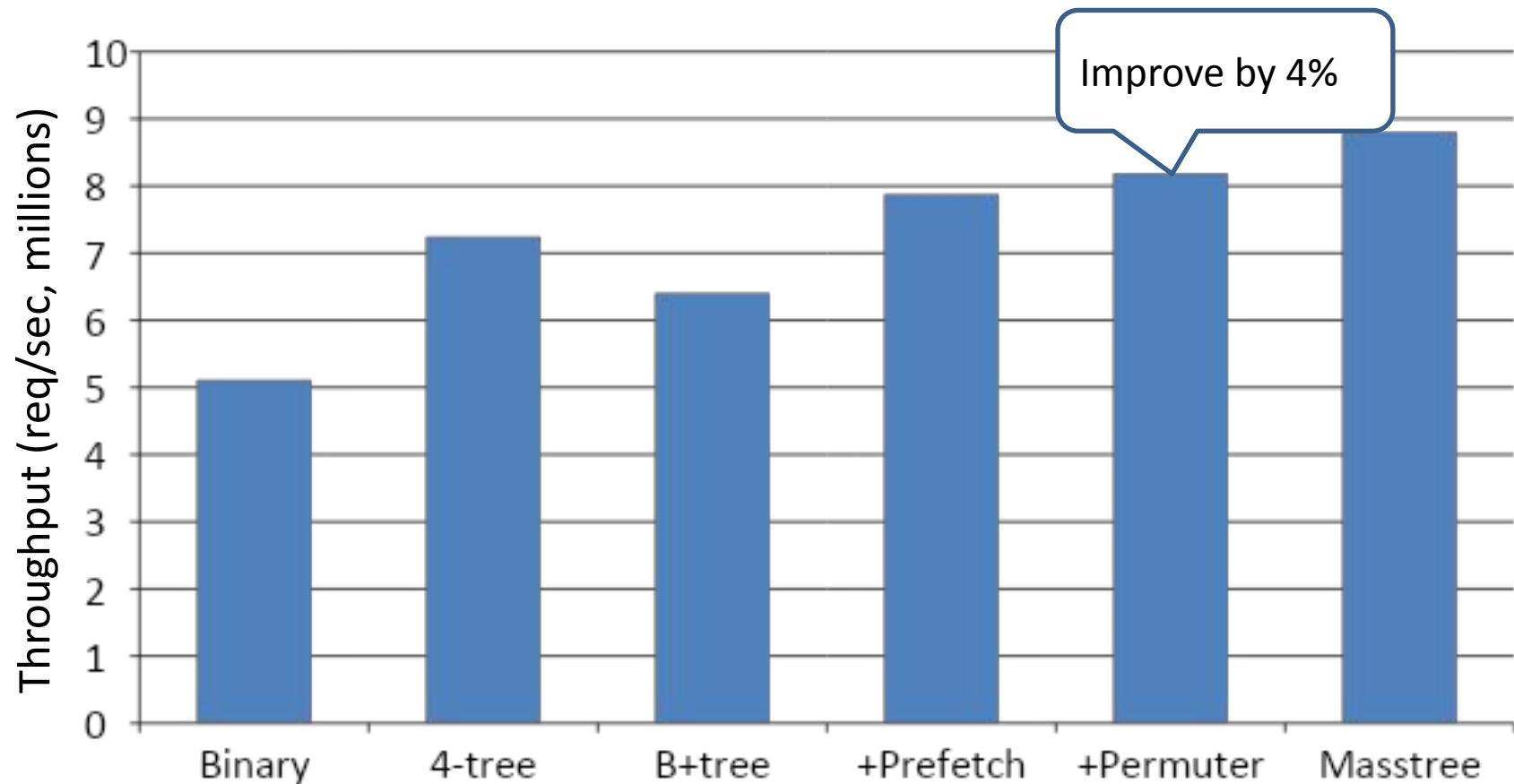
Permuter: 64-bit integer



- A concurrent lookup does not need to retry
 - Lookup uses permuter to search keys
 - Insert appears atomic to lookups

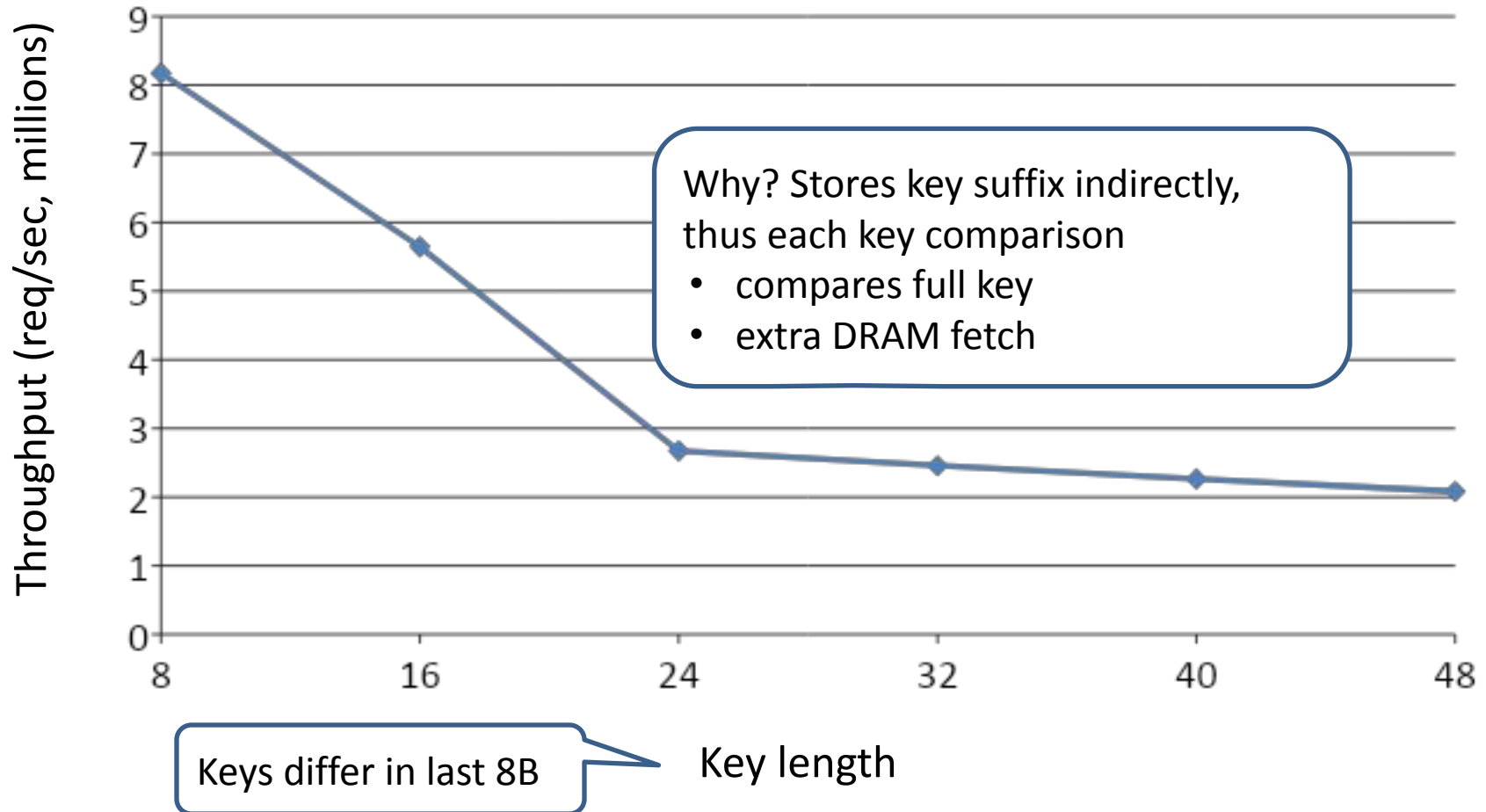
B+tree with permuter

140M short KV, put-only, @16 cores



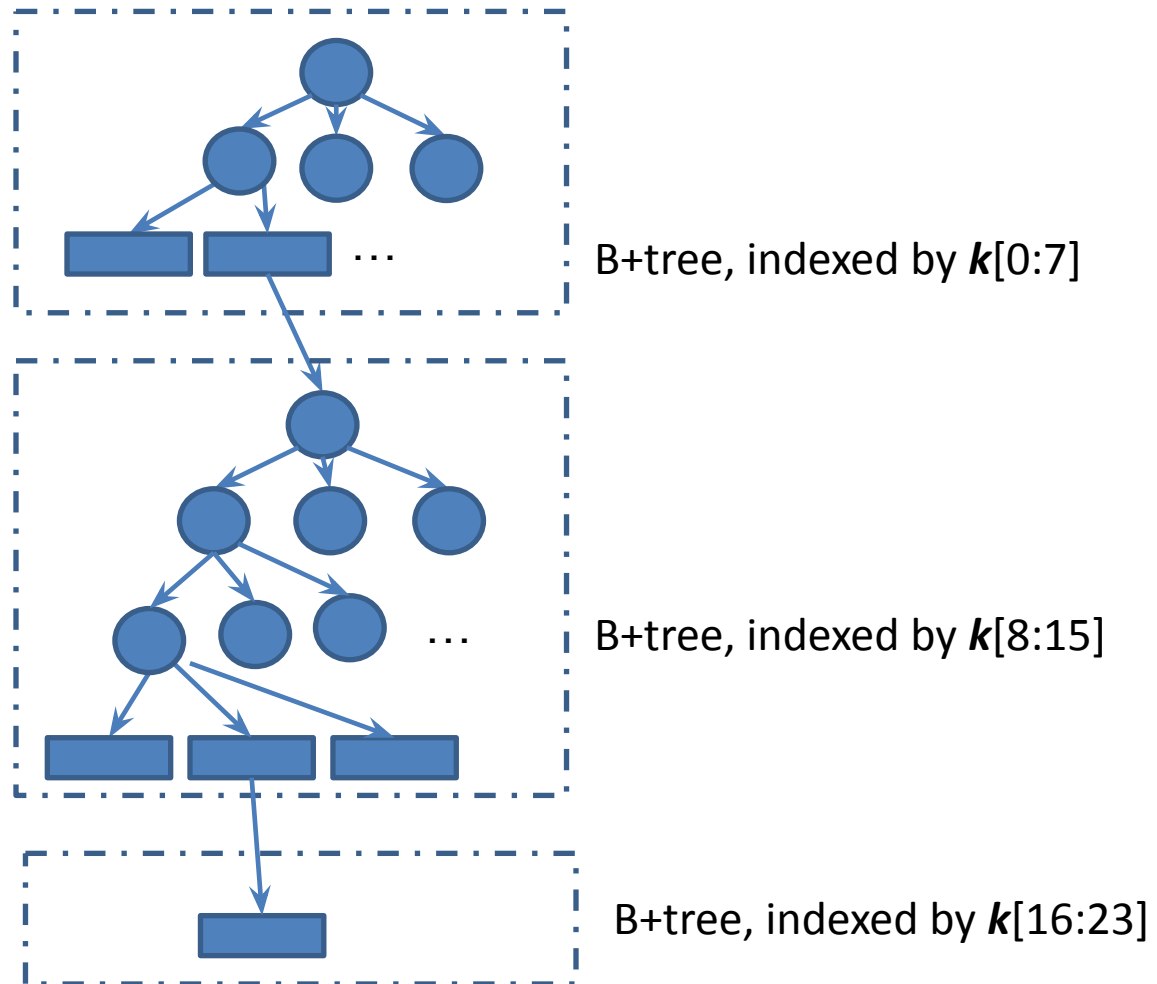
Performance drops dramatically when key length increases

Short values, 50% updates, @16 cores, no logging

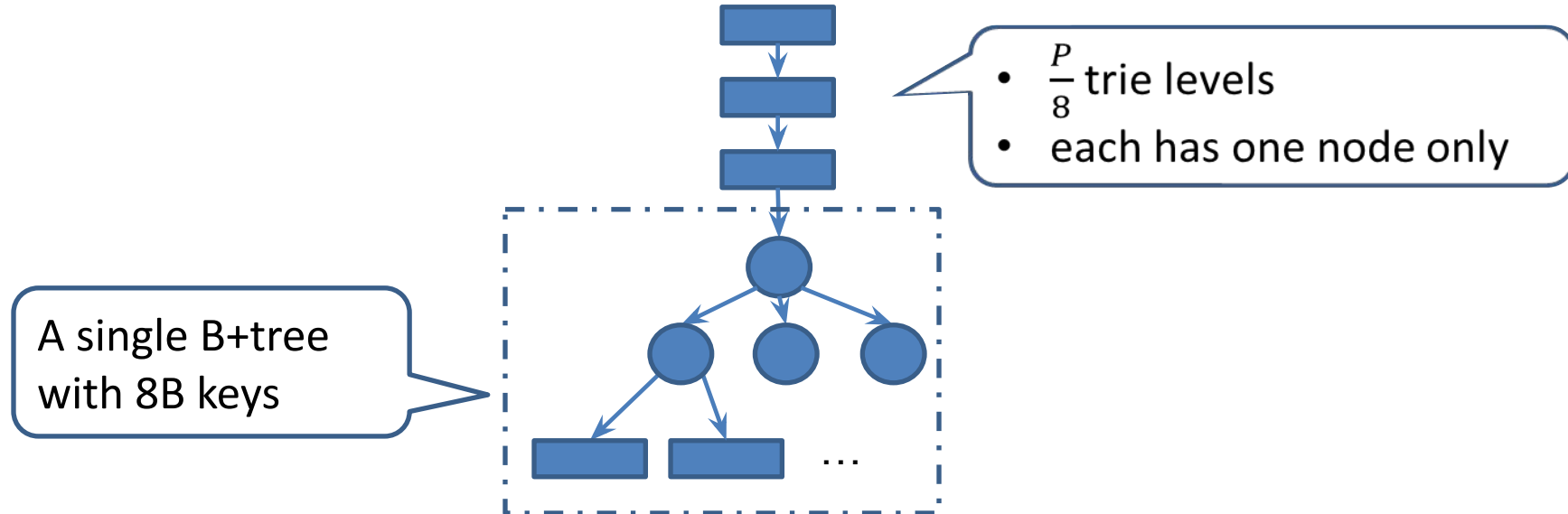


Masstree – Trie of B+trees

- Trie: a tree where each level is indexed by fixed-length key fragment
- Masstree: a trie with fanout 2^{64} , but each trie node is a B+tree
- Compress key prefixes!



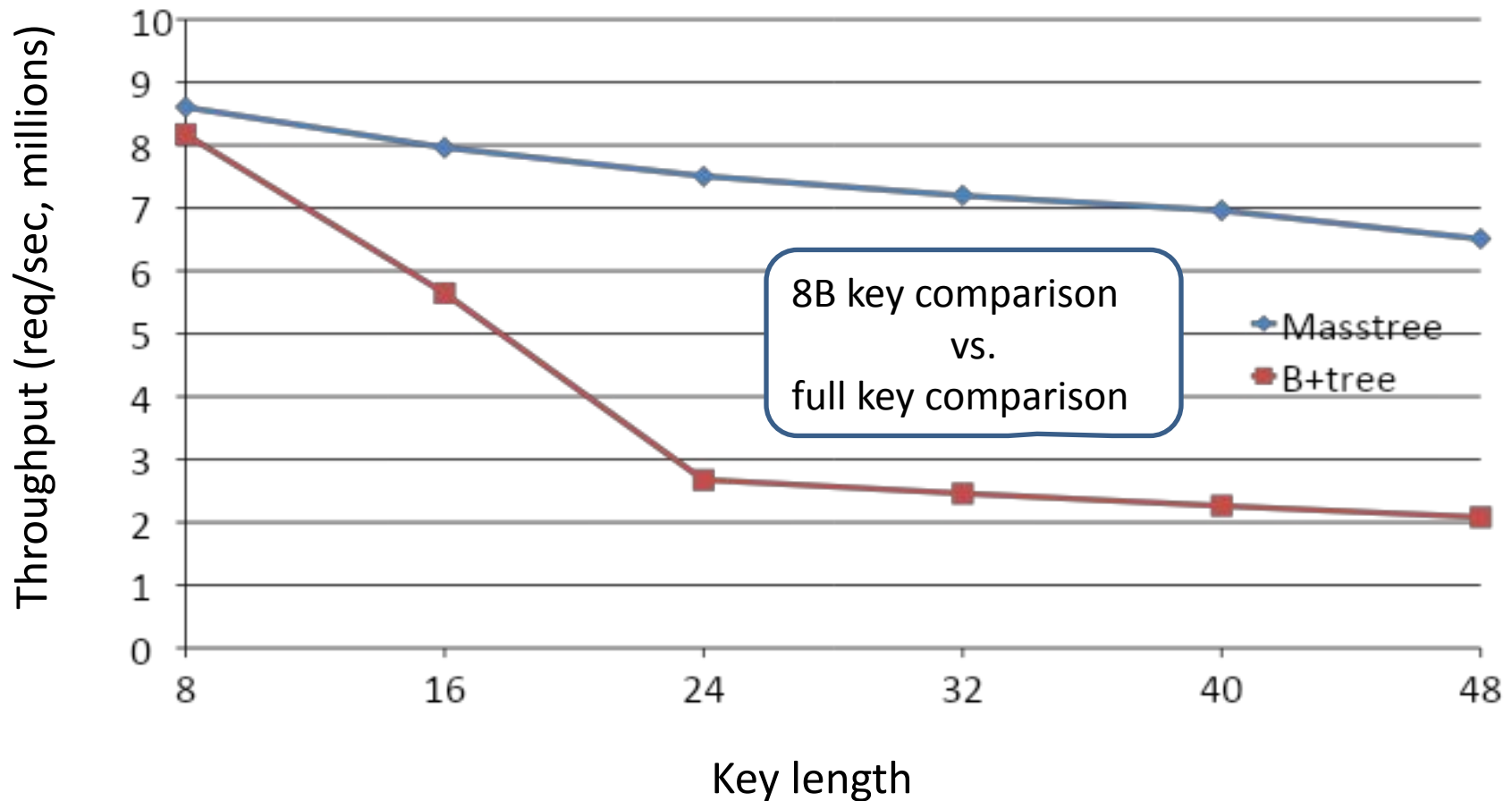
Case Study: Keys share P byte prefix – Better than single B+tree



	Complexity	DRAM access
Masstree	$O(\log N)$	$O(\log N)$
Single B+tree	$O(P \log N)$	$O(P \log N)$

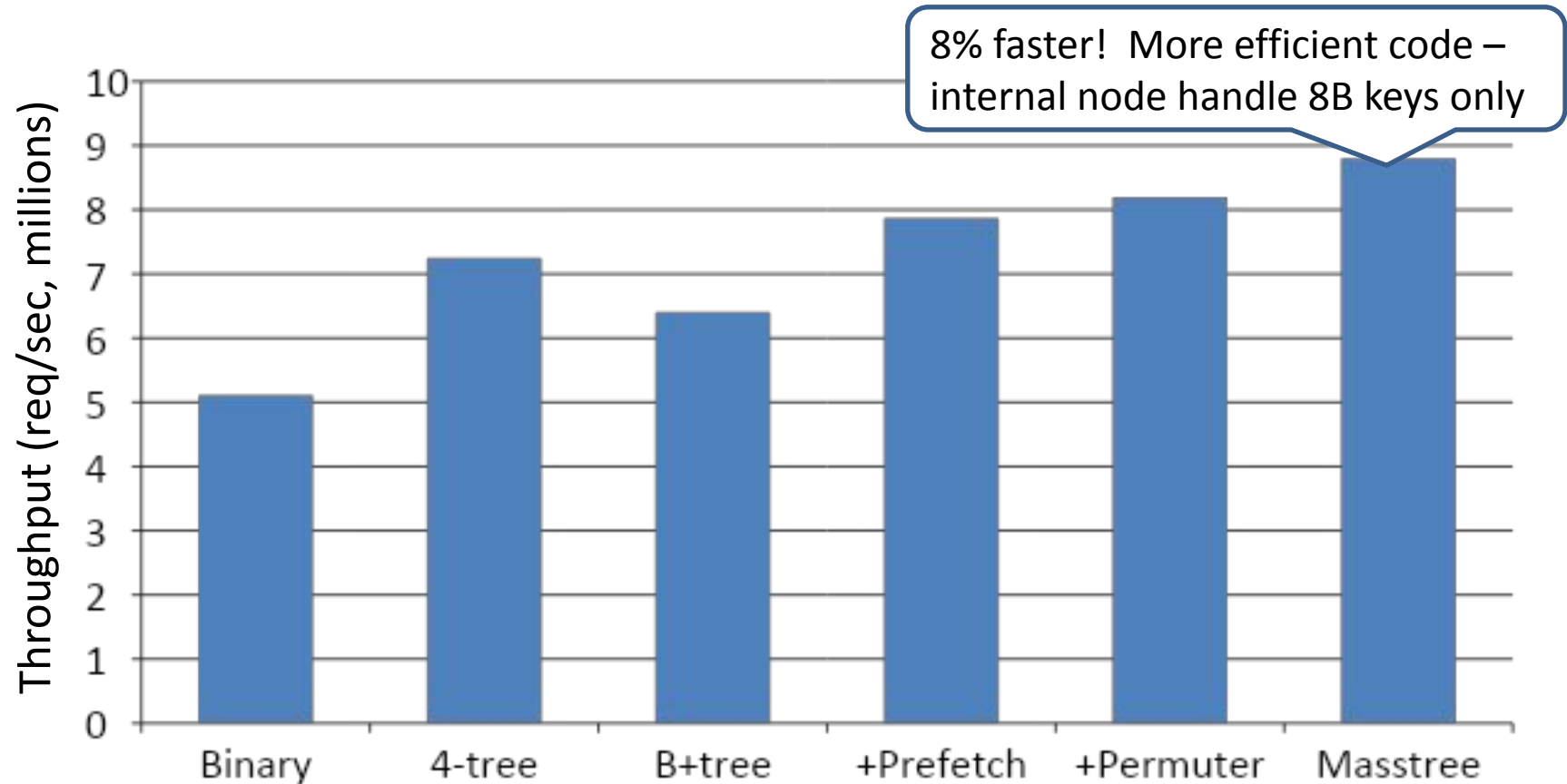
Masstree performs better for long keys with prefixes

Short values, 50% updates, @16 cores, no logging



Does trie of B+trees hurt short key performance?

140M short KV, put-only, @16 cores

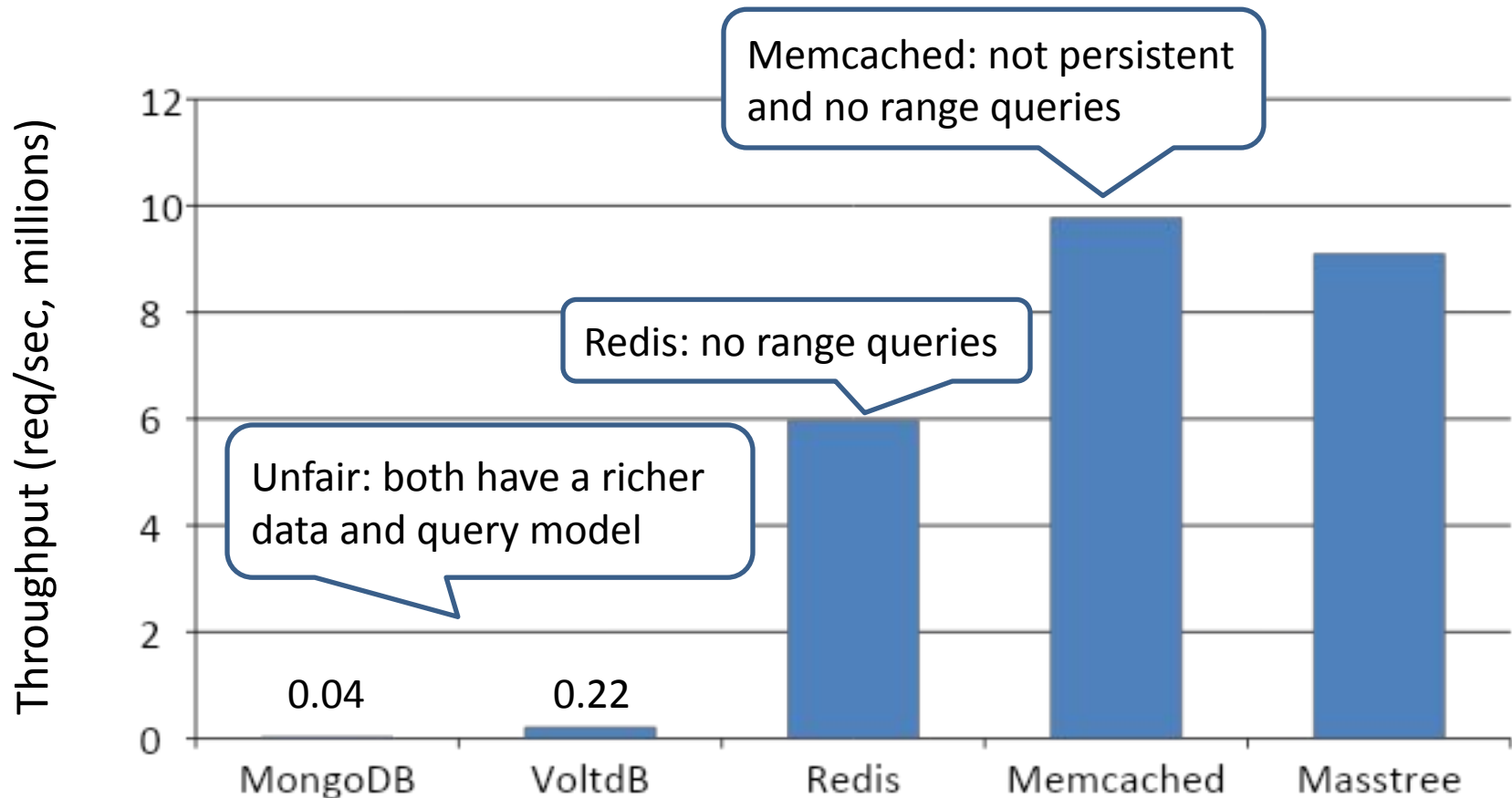


Evaluation

- Masstree compare to other systems?
- Masstree compare to partitioned trees?
 - How much do we pay for handling skewed workloads?
- Masstree compare with hash table?
 - How much do we pay for supporting range queries?
- Masstree scale on many cores?

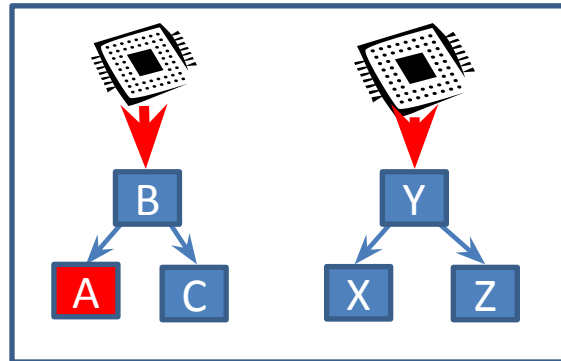
Masstree performs well even with persistence and range queries

20M short KV, uniform dist., read-only, @16 cores, w/ network

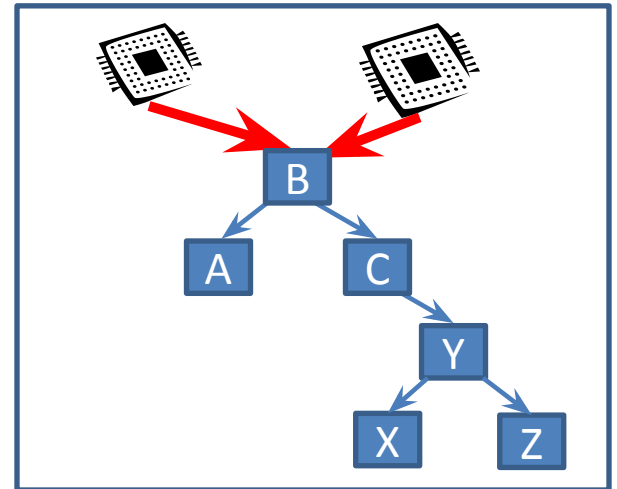


Multi-core – Partition among cores?

- Multiple instances, one unique set of keys per inst.
 - Memcached, Redis, VoltDB

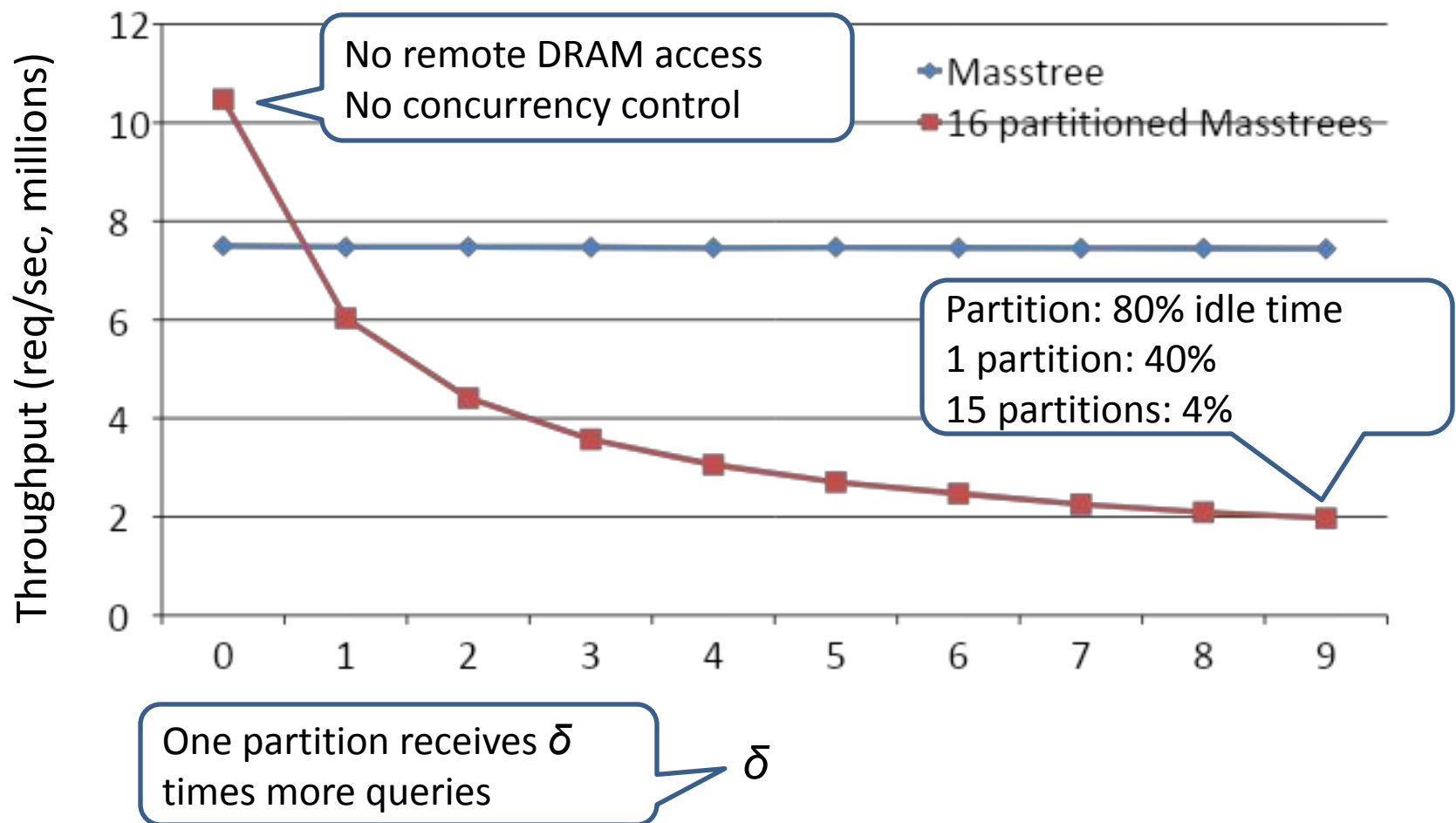


- Masstree: a single shared tree
 - each core can access all keys
 - reduced imbalance



A single Masstree performs better for skewed workloads

140M short KV, read-only, @16 cores, w/ network

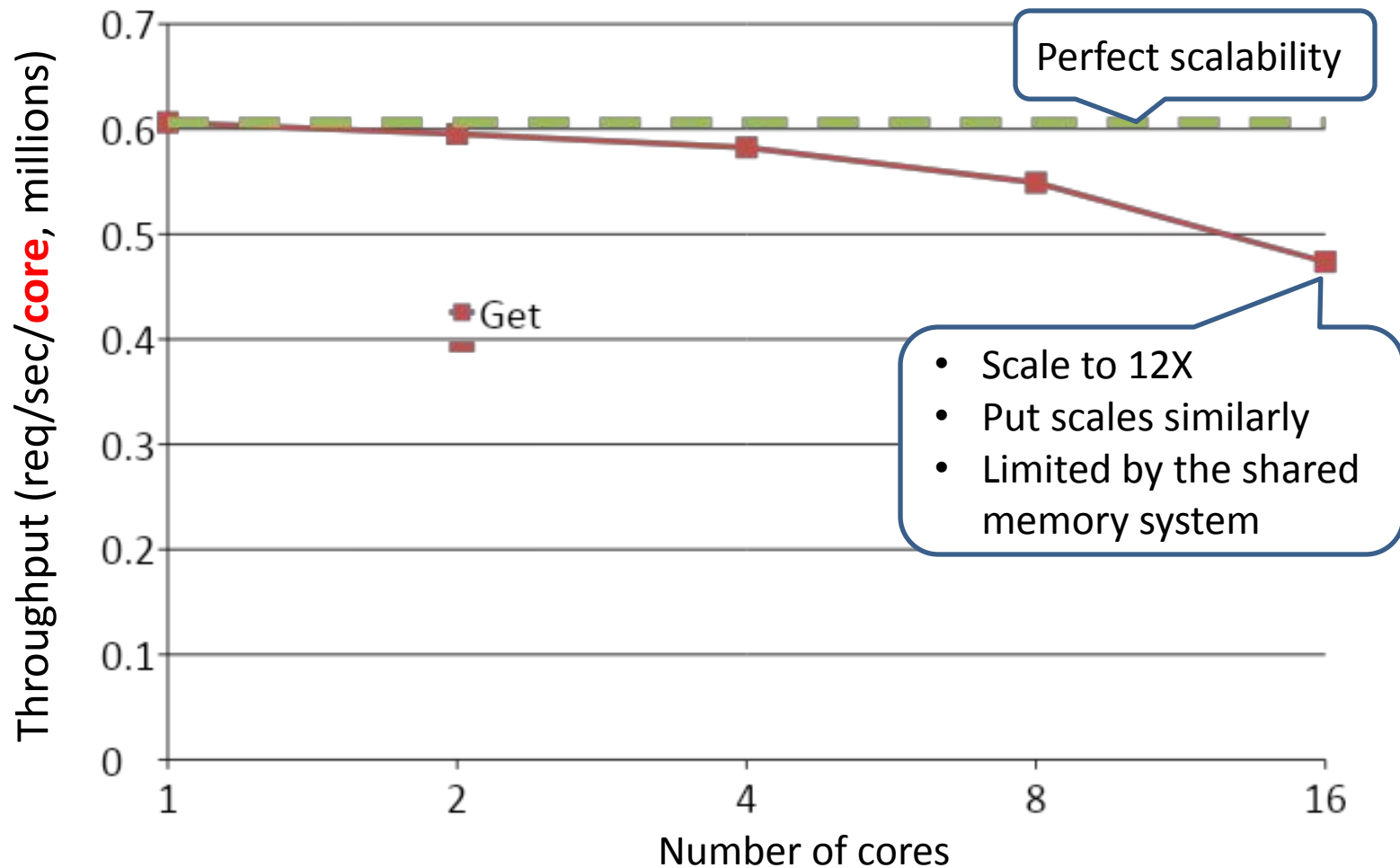


Cost of supporting range queries

- Without range query? One can use hash table
 - No resize cost: pre-allocate a large hash table
 - Lock-free: update with cmpxchg
 - Only support 8B keys: efficient code
 - 30% full, each lookup = 1.1 hash probes
- Measured in the Masstree framework
 - 2.5X the throughput of Masstree
- Range query costs 2.5X in performance

Scale to 12X on 16 cores

Short KV, w/o logging



Related work

- [OLFIT]: Optimistic Concurrency Control
- [pB⁺-trees]: B+tree with software prefetch
- [pkB-tree]: store fixed # of diff. bits inline
- [PALM]: lock-free B+tree, 2.3X as [OLFIT]
- Masstree: first system combines them together, w/ new optimizations
 - Trie of B+trees, permuter

Summary

- Masstree: a **general-purpose high-performance** persistent KV store
- 5.8 million puts/sec, 8 million gets/sec
 - More comparisons with other systems in paper
- Using cache-craftiness improves performance by 1.5X

Thank you!